MVP 1 is the complete hardware layer: one Arduino UNO Q reads the office and data-center sensors, controls their LEDs and servo mechanisms, and exchanges structured data with the backend.

It should work independently of Nemotron, Photon, and the finished dashboard. That lets the rest of the team build against a stable hardware interface.

1. What we are building

Component	Office	Data center	Total
Arduino UNO Q	Shared	Shared	**1**
Grove light sensor	1	—	**1**
Flame/IR sensor	—	1	**1**
SG90 servo representing a fan	1	1	**2**
White LEDs	3	—	**3**
Green LEDs	—	3	**3**
1 kΩ LED resistors	3	3	**6**
Breadboards	As needed	As needed	Up to **2**
Jumper wires	As needed	As needed	As needed

Also needed:

- Suitable UNO Q power and a USB-C data cable.
- External power suitable for the two servos.
- A Grove cable or Grove-to-jumper breakout for the light sensor.
- Servo horns, mounting screws, and lightweight attachments.
- The two building models.
- Ideally, access to a multimeter for checking power and sensor output voltages.

Temperature sensors are outside the required version until their actual models are confirmed. There are no doors, tap modules, switches, water sensors, or humidity sensors.

2. What this MVP must accomplish

By the end, we should be able to:

1. Read office illumination.
2. Read the data-center flame/IR signal.
3. Identify which building produced each reading.
4. Turn each of the six LEDs on or off independently.
5. Start or stop each servo’s demonstration movement independently.
6. Forward readings to the backend.
7. Receive backend commands and return acknowledgments.

8. Detect loss of communication with the controller.

The two buildings share a controller but remain separate in the data.

3. Electrical requirements

UNO Q is not the same electrical platform as an Uno R3. Its maker-header GPIO uses 3.3 V logic. Analog inputs must remain within their permitted 3.3 V range; do not feed a 5 V analog output directly into them. [UNO Q power and I/O specifications](https://docs.arduino.cc/tutorials/uno-q/power-specification/)

Use this arrangement:

Device	Power arrangement
UNO Q	A stable supply suitable for the board. Arduino recommends **5 V at 3 A** for USB-C power.
Grove light sensor	Use **3.3 V** for the compatible Grove Light Sensor shown in your photo.
Flame/IR module	Confirm its permitted supply voltage and output levels before connecting it.
Servos	External regulated power within the supplied servos' ratings, sized for both servos' current demands.
LEDs	Individual GPIO connections through individual resistors.

The UNO Q itself can require more power than an older Uno. Don't assume an arbitrary laptop USB port supplies its recommended power. [Arduino's power guidance](https://docs.arduino.cc/tutorials/uno-q/power-specification/)

For the servos:

- External positive supply connects to servo power.
- External ground connects to servo ground.
- External ground also connects to UNO Q ground.
- Servo signal wires connect to the assigned GPIO pins.

For this wiring plan, the external servo supply's positive lead goes to the servos—not into the UNO Q's power pins.

4. Proposed pin allocation

This is a **proposed allocation to bench-test**, not a claim that the complete circuit has already been verified.

UNO Q pin	Component	Building
D2	White LED 1	Office
D3	White LED 2	Office
D4	White LED 3	Office
D5	Green LED 1	Data center
D6	Green LED 2	Data center
D7	Green LED 3	Data center
D8	Flame module digital output, after voltage verification	Data center
D9	Servo signal	Office
D10	Servo signal	Data center
A0	Grove light-sensor analog output	Office
A1	Reserved for optional flame analog output, after voltage verification	Data center

Leave other pins unused initially. This fits within the exposed UNO Q headers. [Official pinout](https://docs.arduino.cc/resources/pinouts/ABX00162-full-pinout.pdf)

The exact flame module's **supply, output polarity, and connector order remain unresolved**. "Boochen3296" alone is insufficient to finalize those connections.

5. LED wiring and control

Wire each two-leg LED separately:

GPIO → 1 kΩ resistor → LED anode → LED cathode → ground

Each LED gets its own resistor. Don't share a single resistor across three parallel LEDs.

Independent control is useful because we can:

- Turn all office lights on.
- Turn one office light off.
- Restore it.
- Control data-center indicators separately.

At 3.3 V, some white LEDs may be dim with 1 kΩ resistors. Test whether the Grove sensor can distinguish the lighting changes before permanently mounting everything. Adjust placement and shielding first; any resistor change should respect the actual LED and GPIO ratings.

6. Office light sensing

The photographed Grove Light Sensor produces an analog voltage related to illumination. Seeed specifies a 3–5 V operating range; powering this version at 3.3 V keeps its output within the intended board voltage domain. Its reading is relative illumination, not a calibrated lux

measurement. [Grove Light Sensor documentation](https://wiki.seeedstudio.com/Grove-Light_Sensor/)

For the standard Grove cable arrangement:

Grove connection	Proposed destination
VCC	UNO Q 3.3 V
GND	Ground
Analog signal	A0
Unused connector position	Leave disconnected

Verify the module labels and cable mapping before powering it.

Mount the sensor:

- Facing the office LEDs.
- At a fixed distance.
- Where its view will remain consistent.
- With a small hood or cardboard shield if ambient room lighting dominates.

A roofless model is fine, but inconsistent surrounding light can make the demonstration unreliable.

Calibration procedure

1. Record roughly ten seconds with the office LEDs off.
2. Record roughly ten seconds with all three on.
3. Switch individual LEDs off and on.
4. Confirm that the readings change reproducibly.
5. Save the normal lighting baseline and ADC configuration.

Do not assume a universal threshold such as “below 500 means broken.”

If a single failed LED produces too little contrast, the MVP can demonstrate a larger lighting reduction. The sensor cannot independently identify which LED failed.

7. Data-center flame/IR sensing

For the first version, use the module’s **digital output** if its electrical compatibility is confirmed.

Collect:

- Raw digital level.

- A normalized `ir_alert` value.
- Whether an alert occurred during the reporting interval.
- The time the reading was received.

Determine experimentally whether the output is active-low or active-high. Don’t assume that `LOW` always means an alert.

If the module also exposes an analog output, add it only after confirming that its voltage stays within the UNO Q analog-input limit. It is optional for MVP 1.

For testing, use a suitable, verified IR test source. If you use a software-injected test event, label it as simulated. A flame/IR signal does not establish that an actual fire exists, and no live flame is needed for this project.

8. Servo behavior

Use one SG90 per building to represent equipment activity.

Command	Demonstration behavior
Running	Repeatedly sweep a lightweight arm through a tested, limited range.
Stopped	Stop the repeated sweep and hold a defined position.

Start with a modest range, such as approximately 60°–120°, and verify the actual mechanism before widening it. Don’t force the servo against a mechanical stop.

Use nonblocking timing so movement doesn’t prevent sensor sampling or command handling.

The current Arduino Servo library includes a Zephyr implementation, but we still need to compile and test it with the installed UNO Q board package. [Official Servo implementation](https://github.com/arduino-libraries/Servo/blob/master/src/zephyr/Servo.cpp)

Important: a stopped sweep may still hold position and draw current. It is not the same as disconnecting power.

The hardware reports:

- `fan_command: running`
- `fan_command: stopped`

It does **not** report measured RPM, actual position, actual airflow, or verified servo health.

9. UNO Q software structure

There are two small programs:

Program	Runs where	Responsibility
Arduino sketch	UNO Q microcontroller	Sample sensors, operate LEDs and servos, maintain commanded states.
Python adapter	UNO Q Linux processor	Exchange data with the sketch and forward it to the Node backend.

Use Arduino's supported **Bridge** mechanism between the microcontroller and Python. Bridge already uses internal communication resources, so avoid repurposing that internal link. [Bridge API reference](https://docs.arduino.cc/software/app-lab/bridge/bridge-api/)

The proposed external connection is ordinary HTTP over a reachable local network:

Sketch ↔ Bridge ↔ Python adapter ↔ Node backend

USB-C is used for initial setup and programming; the board's power arrangement must meet its requirements.

For MVP 1, a tiny test receiver can stand in for the full backend.

10. Responsibilities of the sketch

The sketch should:

- Initialize known GPIO states.
- Keep LEDs off and servo sweeping disabled until commanded.
- Sample office illumination regularly.
- Sample the IR digital input often enough to capture the demonstration.
- Maintain a snapshot of readings and commanded states.
- Count IR activations or latch a brief activation until it is reported.
- Accept a small, explicit set of hardware commands.
- Return acknowledgments.
- Continue operating without blocking on long delays.

Suggested starting rates—not fixed sensor specifications:

Operation	Starting target
Light sampling	About every 100 ms
IR digital sampling	About every 10–20 ms
Snapshot forwarding	About once per second
Frontend refresh	Every two seconds, handled later

Longer-running work such as network requests belongs in the Linux adapter, not the sensor-control loop.

11. Responsibilities of the adapter

The adapter should:

1. Obtain the current sketch snapshot through Bridge.
2. Add a reception timestamp.
3. Forward the snapshot to the backend.
4. Retrieve pending hardware commands.
5. Execute allowed commands through Bridge.
6. Send command acknowledgments.
7. Retry temporary network failures without crashing.
8. Keep logs explaining connection problems.

Use one fixed device identifier, for example **`unoq-main`**.

Within that device, use fixed building identifiers:

- **`office`**
- **`data_center`**

Don’t identify buildings by the order in which readings arrive.

12. Shared data contract

Agree on the format before the hardware and backend teams work separately.

Each snapshot should contain:

Field	Meaning
Schema version	Which message format is being used.
Device ID	The shared UNO Q.
Sequence number	Detect missing or repeated snapshots.
Microcontroller uptime	Help identify restarts.
Reception timestamp	Added by the adapter/backend.
Building ID	Office or data center.
Sensor ID and raw value	Identify the measurement.
ADC configuration	Interpret the analog reading correctly.
Source	Hardware or simulated test input.
Commanded equipment states	Kept separate from measurements.

Illustrative snapshot—not actual readings:

```json
{
  "schema_version": 1,
  "device_id": "unoq-main",
  "sequence": 42,
  "mcu_uptime_ms": 120000,
  "source": "hardware",
  "buildings": {
    "office": {
      "sensors": {
        "light": {
          "raw": 620,
          "unit": "adc_counts"
        }
      },
      "commanded": {
        "leds": [true, true, true],
        "fan": "running"
      }
    },
    "data_center": {
      "sensors": {
        "flame_ir": {
          "digital_level": 1,
          "ir_alert": false,
          "alert_seen_since_last_report": false
        }
      },
      "commanded": {
        "leds": [true, true, true],
        "fan": "stopped"
      }
    }
  }
}
```

The flame boolean must be normalized using the **tested polarity**. The sample value above is not a wiring assumption.

13. Supported commands

Keep commands limited and predictable.

Command	Required information
Set LED	Command ID, building, LED number, on/off
Set fan representation	Command ID, building, running/stopped
Get snapshot	Device ID

Validate building names, LED numbers, and supported actions.

An acknowledgment should contain:

- Command ID.
- Whether the command was accepted.
- Resulting commanded state.
- Error information if rejected.

An acknowledgment means the controller applied the instruction in software. It does not prove physical movement.

Proposed backend handoff routes could be:

- `POST /api/hardware/telemetry`
- `GET /api/hardware/commands`
- `POST /api/hardware/commands/:id/ack`

These are **our proposed routes**, not existing Arduino endpoints. The backend teammate can change them if everyone agrees.

14. Connection and sensor-status limits

Use a heartbeat or snapshot timeout. For example, with one-second reporting, five seconds without a new snapshot could mark the device stale.

Because there is one controller, losing it affects both buildings.

Also distinguish:

- **Device connected:** the adapter is receiving fresh snapshots.
- **Reading sampled:** the microcontroller obtained an input value.
- **Sensor physically verified:** requires actual diagnostic evidence.

An unplugged analog sensor can produce a floating numeric reading. A disconnected digital sensor can also appear high or low. Fresh samples alone do not prove that every sensor is healthy.

15. Build and test order

Step	Test	Expected result
1	Start UNO Q and run a basic App Lab example	Board starts reliably.
2	Connect one LED with its resistor	On/off command works.
3	Connect all six LEDs	Each can be controlled independently.
4	Connect Grove sensor at the verified voltage	Light and shade produce repeatable changes.
5	Connect verified flame module	Test input produces the expected raw and normalized states.
6	Connect one externally powered servo	Limited sweep and stop work without resets.
7	Add the second servo	Each responds only to its own command.
8	Combine sketch functions	Sensors keep reporting while servos move.
9	Connect Bridge and adapter	Structured snapshots reach the test receiver.
10	Send backend commands	Correct outputs respond and acknowledgments return.
11	Interrupt networking	Device becomes stale; adapter handles recovery.
12	Restart the board	Uptime resets, communication recovers, startup states are predictable.

Don’t permanently mount everything until the loose-wired version passes these checks.

16. Handoff deliverables

The person responsible for MVP 1 should provide:

- Working UNO Q sketch.
- Working Python adapter.
- Final verified pin map.
- Confirmed module names, voltages, and polarity.
- LED and light-sensor calibration notes.
- Sample hardware snapshots.
- Supported command format.
- A short setup-and-run guide.
- A labeled replay file or simulator for software teammates.
- A list of unresolved hardware limitations.

MVP 1 is complete when real office light readings and data-center IR readings reach the test receiver, all six LEDs and both servo mechanisms respond to correctly targeted commands, and communication loss is visible.

Nemotron investigation, owner approval, Photon calls, and the polished dashboard belong to the other MVPs.